

Common Problems with Procedural Implementation

The examples shown in this chapter share a common problem: all the data the functions operate on is stored in one or more global variables. For the following reasons, using lots of global data with procedural programming is bad coding practice:

1. Any function that uses and/or changes global data cannot easily be reused in a different program. A function that accesses global data is operating on data that lives at a different (higher) level than the code of the function itself. That function will need a `global` statement to access this data. You can't just take a function that relies on global data and reuse it in another program; it can only be reused in a program with similar global data.
2. Many procedural programs tend to have large collections of global variables. By definition, a global variable can be used or changed by any piece of code anywhere in the program. Assignments to global variables are often widely scattered throughout procedural programs, both in the main code and inside functions. Because variable values can change anywhere, it can be extremely difficult to debug and maintain programs written this way.
3. Functions written to use global data often have access to too much data. When a function uses a global list, dictionary, or any other global data structure, it has access to *all* the data in that data structure. However, typically the function should operate on only one piece (or just a small amount) of that data. Having the ability to read and modify any data in a large data structure can lead to errors, such as accidentally using or overwriting data that the function was not intended to touch.

Object-Oriented Solution—First Look at a Class

[Listing 1-7](#) is an object-oriented approach that combines all the code and associated data of a single account. There are many new concepts here, and